Arquitectura de Sistemas y Estrategia de Desarrollo Agéntico para Plataformas de Terapia Holística de Alto Rendimiento

1. Visión Estratégica y Contexto del Mercado de Bienestar Digital

La intersección entre la tecnología de consumo y la espiritualidad moderna —a menudo denominada "Tech-Mysticism" o "Wellness Economy"— ha evolucionado desde blogs amateurs y tiendas básicas de Etsy hacia ecosistemas digitales complejos que mueven miles de millones de dólares anualmente. Cuando se plantea el objetivo de construir una plataforma "como lo hacen los grandes desarrolladores", no se hace referencia únicamente a la estética visual, sino a la robustez de la ingeniería subyacente que permite escalar operaciones, gestionar la confianza del usuario y procesar transacciones híbridas (servicios intangibles y productos físicos) sin fricción.

Los desarrolladores de élite y las agencias de producto digital no utilizan constructores de sitios genéricos para estos proyectos. En su lugar, implementan arquitecturas de **Comercio Componible (Composable Commerce)** y **Headless**, desacoplando la experiencia de usuario (frontend) de la lógica de negocio (backend). Este reporte establece una hoja de ruta técnica exhaustiva, diseñada para ser ejecutada mediante ingeniería agéntica (AI-driven development) utilizando herramientas como Google Antigravity, para replicar los estándares de calidad de líderes del mercado como *The Hoodwitch*, *Labyrinthos Academy* y *Sage Goddess*.

El desafío central de este proyecto radica en la **naturaleza híbrida del inventario**. A diferencia de un e-commerce tradicional que vende zapatillas (producto físico) o una plataforma de reservas que vende citas (servicio), una plataforma de terapia holística debe manejar ambos simultáneamente. Un usuario puede desear agendar una "Lectura de Carta Astral" (que requiere gestión de calendario y entrega digital) y comprar un "Kit de Limpieza Energética" (que requiere cálculo de envío y gestión de stock) en una sola transacción unificada. Resolver este "Carrito Mixto" con elegancia es la marca distintiva de un desarrollo de nivel empresarial.[1]

1.1. El Estándar de Experiencia de Usuario (UX) en el Nicho Místico

El análisis de sitios de referencia revela que el éxito no solo depende de la funcionalidad, sino de la "atmósfera digital". Los usuarios de este nicho buscan validación, calma y conexión.

- **Confianza y Autoridad:** Sitios como *The Hoodwitch* [3] han logrado convertir el

esoterismo en un estilo de vida aspiracional mediante una dirección de arte que rivaliza con revistas de moda. La tecnología debe servir a esta estética: tiempos de carga instantáneos, transiciones suaves y micro-interacciones que evocan "magia" (ej. efectos de partículas sutiles, *hover states* etéreos).
- **Contenido como Motor de Comercio:** La venta no es directa; es contextual. El sistema debe permitir relacionar semánticamente contenido educativo (artículos sobre fases lunares) con productos comerciales (velas rituales), creando un embudo de conversión orgánico.[4]

---

2. Deconstrucción de Referentes: Ingeniería Inversa de los Líderes

Para instruir a un agente de IA sobre qué construir, es imperativo desglosar funcionalmente a los referentes del mercado. No basta con copiar su apariencia; se debe replicar su lógica operativa.

2.1. The Hoodwitch: Arquitectura de Contenido y Comercio

Este sitio [3] ejemplifica la integración de **Content-First Commerce**.

- **Modelo de Negocio:** Venta de parafernalia mística (física) y contenido editorial (digital).
- **Arquitectura Inferida:** Utiliza un CMS robusto (probablemente Squarespace o una solución Headless personalizada) que permite incrustar "bloques de producto" dentro de narrativas largas.
- **Lección para el Desarrollo:** El sistema de gestión de contenidos (CMS) que se solicite al agente de IA debe soportar *Relaciones Polimórficas*. Un artículo de blog debe tener un campo de "Productos Relacionados" que renderice tarjetas de compra directamente en el flujo de lectura, reduciendo la fricción para el usuario que se siente inspirado por el artículo a realizar una compra inmediata.[6]

2.2. Labyrinthos Academy: Gamificación y Aplicaciones Web Progresivas (PWA)

Labyrinthos [7] redefine la educación del tarot mediante tecnología.

- **Funcionalidad Distintiva:** Aplicaciones de lectura de tarot integradas. No son simples generadores de números aleatorios; son sistemas que simulan la física de un mazo real y ofrecen interpretaciones basadas en bases de datos complejas de significados de cartas.
- **Estrategia de Retención:** Gamificación del aprendizaje. Los usuarios tienen perfiles que rastrean su progreso en cursos de tarot.
- **Lección para el Desarrollo:** Se requiere una arquitectura de **Single Page Application (SPA)** o **PWA** para que las interacciones complejas (barajar cartas, seleccionar tiradas)

ocurran instantáneamente en el navegador sin recargar la página. Esto dicta la elección de tecnologías como React o Next.js para el frontend.[7]

2.3. Sage Goddess: Gestión de Inventario y Comunidad a Escala

Este sitio [9] maneja un volumen masivo de SKUs (Stock Keeping Units) y eventos en vivo.

- **Funcionalidad Distintiva:** Programas de membresía y gestión de inventario de piezas únicas (cristales que son irrepetibles).
- **Lección para el Desarrollo:** El backend debe manejar **Control de Concurrencia** estricto. Si dos usuarios intentan comprar el mismo cristal único al mismo tiempo, el sistema debe bloquear el ítem para el primero que inicia el checkout y liberarlo solo si la transacción falla. Esto requiere bases de datos transaccionales con propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).[11]

---

3. Arquitectura Técnica: El Paradigma "Headless" y "Composable"

Para construir "como los grandes", se debe abandonar la idea de una plantilla monolítica (todo en uno). La arquitectura recomendada es **Headless Commerce**, donde el frontend (la cabeza) se desacopla del backend (el cuerpo). Esto permite iterar el diseño sin romper la base de datos y conectar múltiples servicios especializados (pagos, envíos, reservas) vía API.

3.1. Stack Tecnológico Recomendado para Ingeniería Agéntica

Al utilizar Google Antigravity u otros agentes de desarrollo, la precisión en la definición del stack es crucial para evitar alucinaciones técnicas o código obsoleto.

Capa del Stack	Tecnología Recomendada	Justificación Técnica para el Agente de IA
Frontend (Framework)	**Next.js 15 (App Router)**	Estándar de la industria. Permite *Server-Side Rendering (SSR)* para SEO (vital para búsquedas como "terapia reiki") y *React Server Components* para rendimiento óptimo. Soporta PWAs nativamente.[13]

Lenguaje de Programación	**TypeScript**	Provee tipado estático, reduciendo errores en tiempo de ejecución y facilitando el mantenimiento a largo plazo. Es el lenguaje preferido para proyectos escalables.[15]
Estilos y UI	**Tailwind CSS + Framer Motion**	Tailwind permite un desarrollo rápido de sistemas de diseño consistentes. Framer Motion es esencial para las micro-interacciones "místicas" y animaciones fluidas que definen la estética de lujo.[16]
Base de Datos y Auth	**Supabase (PostgreSQL)**	Ofrece una base de datos relacional robusta (necesaria para relaciones complejas entre usuarios, citas y productos), autenticación segura y capacidades de tiempo real, todo accesible vía API.[12]
Motor de Comercio	**MedusaJS** o **Shopify Headless**	MedusaJS es una alternativa *open-source* a Shopify, diseñada específicamente para ser modificada. Permite crear lógica personalizada para productos híbridos (digital/físico) mucho más fácilmente que forzar un monolito.[17]
Gestión de Estado	**Zustand**	Librería ligera para gestionar el estado del

		carrito de compras en el cliente. Menos *boilerplate* que Redux, ideal para manejar la lógica compleja de carritos mixtos.[15]
Infraestructura	**Vercel**	Plataforma de despliegue optimizada para Next.js, con soporte para *Edge Functions* que permiten ejecutar lógica de personalización cerca del usuario geográfico.[13]

3.2. Comparativa: Monolito vs. Headless

Es común considerar plataformas como Shopify o WooCommerce "out of the box". Sin embargo, el análisis comparativo muestra por qué los "grandes" migran a Headless.

Tabla 1: Comparativa de Arquitecturas para Terapia Holística

Característica	Monolito (WooCommerce/Shopify Básico)	Headless (Next.js + APIs)	Impacto en el Proyecto
Personalización UX	Limitada por la plantilla *theme*.	Infinita. Pixel-perfect control.	Permite crear experiencias inmersivas (ej. transiciones de tarot, widgets lunares) imposibles en plantillas estándar.[20]
Velocidad de Carga	Dependiente de plugins pesados. Lento.	Optimizado. Carga solo lo necesario.	Mejora drástica en Core Web Vitals, lo que aumenta el ranking en Google y la conversión móvil.[22]

Lógica Híbrida	Difícil. Requiere plugins que chocan entre sí.	Nativa. Se codifica la lógica exacta.	Permite flujos de checkout diferenciados para citas y productos sin "parches" de software.[2]
Escalabilidad	Se degrada con muchos productos/usuarios.	Escala horizontalmente sin límites.	Soporta picos de tráfico durante lanzamientos o eventos virales sin caerse.[23]
Costo de Mantenimiento	Alto en plugins y actualizaciones constantes.	Medio/Alto en desarrollo inicial, bajo en deuda técnica.	Inversión inicial mayor, pero menor costo operativo y mayor agilidad futura.[22]

---

4. Funcionalidades Críticas: Especificaciones para la Generación por IA

Para que un agente de IA como Google Antigravity genere un sistema funcional, se deben traducir los requerimientos de negocio en **Especificaciones de Ingeniería**. A continuación se detallan los módulos críticos.

4.1. Módulo de Gestión de Identidad y Usuarios (IAM)

El "Control de Usuarios" en una plataforma holística debe ser profundo. No es solo un correo y una contraseña; es el perfil espiritual del cliente.

Requerimientos de Datos (User Schema):

El agente debe diseñar una tabla de usuarios extendida en PostgreSQL que incluya:

- **Datos Core:** UUID, email, password_hash, created_at.
- **Perfil Holístico (JSONB):** birth_date, birth_time, birth_place (para cálculos astrológicos futuros), preferences (intereses: cristales, yoga, reiki).
- **Roles (RBAC):** role ('guest', 'user', 'subscriber', 'admin', 'therapist'). Esto permite crear áreas privadas para terapeutas donde gestionan su disponibilidad.[24]

Funcionalidad de "Área Sagrada" (Dashboard):

El dashboard del usuario no debe parecer una hoja de cálculo administrativa. Debe diseñarse

como un "Altar Digital".

- **Visualización:** Separación clara entre "Mis Objetos" (pedidos físicos) y "Mi Camino" (historial de terapias y próximas citas).
- **Seguridad:** Implementación de **NextAuth.js** para gestionar sesiones seguras, permitiendo login social (Google) para reducir fricción en el registro, pero manteniendo la opción de email para privacidad.[14]

4.2. El Carrito Híbrido y la Lógica de Checkout

Este es el punto de falla más común. Un sistema estándar se confunde si intentas cobrar envío por una "Limpieza de Aura" (servicio) que está en el mismo carrito que una "Vela" (producto).

Lógica de Ingeniería para el Agente:

1. **Polimorfismo de Ítems:** La interfaz CartItem en TypeScript debe aceptar discriminadores de tipo.
   TypeScript
   ```typescript
   type ProductType = 'PHYSICAL' | 'SERVICE' | 'DIGITAL';
   interface CartItem {
     id: string;
     type: ProductType;
     price: number;
     quantity: number;
     // Datos específicos condicionales
     shippingWeight?: number; // Solo si PHYSICAL
     bookingDate?: string;    // Solo si SERVICE
     fileUrl?: string;        // Solo si DIGITAL
   }
   ```
2. **Lógica de Envío Condicional:** En el paso de checkout, el sistema debe ejecutar una evaluación:
   - *Algoritmo:* "Si el carrito contiene al menos un ítem PHYSICAL, requerir dirección de envío y llamar API de logística. Si solo contiene SERVICE o DIGITAL, omitir paso de envío y recolección de dirección física, solicitando solo datos de facturación básicos".[26]
3. **Persistencia:** Utilizar localStorage sincronizado con la base de datos cuando el usuario se loguea, asegurando que un carrito armado en el móvil aparezca en el escritorio (Omnicanalidad).[15]

4.3. Sistema de Pagos: Integración Profunda

Para operar "como los grandes" en Latinoamérica y globalmente, la integración de **Mercado Pago** y **Stripe** debe ser nativa, no a través de redirecciones externas que rompen la

experiencia de marca (siempre que sea posible y seguro).

Estrategia de Mercado Pago (Checkout Pro vs. API):

- **Preferencia de Integración:** Utilizar **Checkout Pro** en modo "modal" o embebido, o **Checkout Bricks**. Esto delega la seguridad PCI-DSS a Mercado Pago pero mantiene al usuario visualmente en el contexto del sitio.[28]
- **Creación de la Preferencia (Backend):** El agente debe generar un *endpoint* API (/api/checkout/create-preference) que reciba el carrito, construya el objeto Preference de Mercado Pago incluyendo:
  - items: Array mapeado de los productos.
  - metadata: Objeto JSON con booking_ids para las citas y user_id. Esto es crucial para la conciliación post-pago.
  - back_urls: Direcciones de retorno para éxito, fallo y pendiente en el propio dominio de la aplicación (ej. misitio.com/checkout/success).[29]
- **Webhooks (La Columna Vertebral):** No confiar en la redirección del usuario para confirmar la compra. El sistema debe escuchar notificaciones asíncronas (Webhooks) de Mercado Pago.
  - *Flujo:* Recibe Webhook -> Verifica firma de seguridad -> Consulta estado del pago -> Si approved:
    1. Decrementa stock en DB para ítems físicos.
    2. Cambia estado de booking de "pendiente" a "confirmado" para servicios.
    3. Envía correos transaccionales confirmando la cita y el envío.[31]

4.4. Logística y Envíos Automatizados

La integración logística automatiza el cálculo de costos y la generación de etiquetas.

- **APIs Locales:** Integrar APIs de **Andreani**, **Correo Argentino** o **Envíopack**.[33]
- **Lógica de Cálculo:** El frontend envía el código postal al backend. El backend consulta la API del proveedor logístico pasando el peso total de los ítems PHYSICAL del carrito y las dimensiones de la caja estimada. El proveedor devuelve el costo, que se suma al total del carrito antes de crear la preferencia de pago.[36]

---

5. Ingeniería Agéntica: Flujo de Trabajo con Google Antigravity

Google Antigravity es una plataforma de desarrollo "agent-first" que permite a los desarrolladores orquestar agentes de IA para planificar y ejecutar código complejo.[37] A continuación, se detalla el procedimiento operativo estándar (SOP) para que el usuario construya esta plataforma utilizando dicha herramienta.

5.1. Configuración del Entorno ("Mission Control")

En lugar de escribir código línea por línea, el usuario asume el rol de Arquitecto de Software. El primer paso es establecer la "Misión" en Antigravity.

Prompt Maestro de Inicialización:

> "Actúa como un Arquitecto de Software Senior especializado en E-commerce Headless y Bienestar. Tu misión es inicializar un repositorio monorepo escalable para una plataforma de terapia holística.
>
> **Stack Tecnológico:**
>
> - Framework: Next.js 15 (App Router, React Server Components).
> - Lenguaje: TypeScript (Strict Mode).
> - Estilos: Tailwind CSS + Framer Motion.
> - Base de Datos: Supabase (PostgreSQL).
> - Estado Global: Zustand.
> - Pagos: Mercado Pago SDK.
>
> **Objetivos Iniciales:**
>
> 1. Configurar la estructura del proyecto con carpetas claras para components, lib (utilidades), hooks, y app (rutas).
> 2. Instalar dependencias clave (lucide-react, zod para validación, date-fns para fechas).
> 3. Generar un archivo README.md detallando la arquitectura.
> 4. Crear un plan de implementación (PLAN.md) dividido en 5 fases: Setup, Database, UI Components, Checkout Logic, Testing.".[37]

5.2. Generación de Módulos Específicos (Prompts Tácticos)

Una vez inicializado el proyecto, se debe instruir al agente módulo por módulo. Intentar generar todo de una vez suele resultar en código incompleto.

A. Generación del Esquema de Base de Datos

Prompt:

> "Genera el script SQL para inicializar la base de datos en Supabase. Necesito las siguientes tablas con claves foráneas y restricciones adecuadas:
>
> - profiles: Extiende la tabla de auth.users. Campos: nombre, fecha_nacimiento, hora_nacimiento, biografía.
> - products: id, titulo, descripcion, precio, stock, tipo (ENUM: 'physical', 'service', 'digital'), imagenes (array).
> - bookings: id, user_id, product_id (servicio), fecha_inicio, fecha_fin, estado

(ENUM: 'pending', 'confirmed', 'cancelled').
- orders: id, user_id, total, estado_pago, direccion_envio (JSON).
- order_items: Relación entre orders y products. Incluye precio_unitario al momento de compra. Asegúrate de aplicar políticas RLS (Row Level Security) para que los usuarios solo puedan ver sus propios datos.".[12]

B. Componentes de UI "Místicos"

Prompt:

> "Crea un componente de React ProductCard.tsx usando Tailwind CSS.
>
> **Requerimientos de Diseño:**
>
> - Estética 'Lujo Místico': Fondo oscuro (bg-slate-900), texto claro (text-slate-100), bordes sutiles dorados (border-amber-500/30).
> - Tipografía: Usa una fuente serif para el título (clase font-serif) y sans-serif para el precio.
> - Interacción: Al hacer hover, la tarjeta debe elevarse suavemente (scale-105) y el borde dorado debe brillar más (border-amber-400).
> - Funcionalidad: Debe aceptar una prop product. Si product.type es 'service', el botón debe decir 'Reservar Cita'. Si es 'physical', debe decir 'Agregar al Carrito'.".[40]

C. Integración de Pagos (Backend)

Prompt:

> "Genera una Server Action de Next.js (actions/checkout.ts) para procesar el pago con Mercado Pago.
>
> **Lógica:**
>
> 1. Recibe el cart y userId.
> 2. Valida el stock de los productos físicos en Supabase.
> 3. Instancia el cliente de Mercado Pago.
> 4. Crea un objeto Preference mapeando los ítems.
> 5. Configura auto_return a 'approved'.
> 6. Retorna la URL de redirección (init_point). **Manejo de Errores:** Envuelve todo en un bloque try/catch y devuelve mensajes de error claros.".[30]

5.3. Iteración y Validación Agéntica

Antigravity permite loops de retroalimentación.

- **Instrucción de Test:** "Ejecuta el comando npm run dev. Abre un navegador headless, navega a la página de inicio, verifica que los productos se rendericen. Intenta agregar uno al carrito y verifica que el contador del carrito en el header se actualice. Genera un

reporte de cualquier error encontrado en la consola.".[39]

---

6. Diseño Visual y Estética: El Lenguaje del Lujo Espiritual

El diseño no es un añadido superficial; es el lenguaje con el que la marca comunica su valor. Los "grandes desarrolladores" trabajan con **Sistemas de Diseño** (Design Systems).

6.1. Paletas de Color y Psicología

Se deben evitar los colores neón o primarios saturados. La paleta debe invocar profundidad y misterio.

- **Fondo Principal:** #1a1a2e (Midnight Blue) o #0f172a (Slate 900). Oscuro, reduce la fatiga visual, evoca el cielo nocturno.
- **Acentos Primarios:** #efc07b (Oro envejecido) o #c07a35 (Cobre). Transmite valor, alquimia, lujo.
- **Textos:** #e2e8f0 (Blanco hueso/Gris claro). Nunca usar blanco puro (#ffffff) sobre negro puro, ya que causa vibración visual.
- **Acentos Etéreos:** #d7c2ee (Lavanda pálido) para estados activos o *hovers*.[40]

6.2. Tipografía

La combinación tipográfica es crítica para la legibilidad y el tono.

- **Títulos (Display):** *Cinzel*, *Playfair Display* o *Cormorant Garamond*. Fuentes con serifa, alto contraste, evocan tradición y seriedad académica/mística.
- **Cuerpo (Body):** *Lato*, *Montserrat* o *Inter*. Sans-serifs geométricas o humanistas, altamente legibles en tamaños pequeños y pantallas móviles.[44]

6.3. Animaciones y Micro-interacciones

El movimiento debe ser lento y fluido (*easing* suave), imitando el humo o el agua, no movimientos robóticos lineales.

- **Herramienta: Framer Motion** para React.
- **Aplicación:** Fade-in suave al cargar páginas. Efecto *parallax* sutil en imágenes de fondo (estrellas que se mueven más lento que el contenido al hacer scroll).

---

7. Funcionalidades Avanzadas ("Factor Wow")

Para destacar, la plataforma debe ofrecer valor más allá de la venta.

7.1. Widgets Místicos Integrados

- **Carta del Día (Tarot):** Implementar un componente que consuma una API de Tarot (como tarot-api.onrender.com o una propia). El usuario hace clic en un mazo virtual, se anima el volteo de una carta y se muestra su significado. Esto incrementa el tiempo en el sitio y el retorno diario de usuarios.[46]
- **Fase Lunar en Tiempo Real:** Un widget en el header que muestra la fase lunar actual (calculada astronómicamente o vía API). Si es Luna Llena, el sitio podría sutilmente cambiar su tema de color o destacar productos de "Carga Energética".[48]

7.2. Cross-Selling Algorítmico

Utilizar la base de datos relacional para sugerencias inteligentes.

- **Lógica:** Si el usuario está viendo un servicio de "Reiki para Ansiedad", el sistema consulta la tabla products buscando ítems etiquetados con tag:ansiedad y type:physical (ej. Amatista, Aceite de Lavanda) y los muestra como "Complementos sugeridos para tu terapia".[50]

---

8. Logística y Operaciones (El Mundo Real)

El software debe conectarse con la infraestructura física.

8.1. Integración de Envíos

No ingresar costos manualmente. Usar APIs.

- **Proveedor:** En Argentina/Latam, **Envíopack** o **Andreani** son estándares.
- **Flujo:**
  1. El usuario ingresa CP en Checkout.
  2. Next.js llama a API Route -> API Logística.
  3. La API Logística devuelve opciones (Estándar, Express, Sucursal).
  4. El usuario selecciona -> Se suma al total del carrito.
  5. Al pagar, el Webhook dispara la llamada a la API Logística para "Generar Etiqueta" y devuelve el PDF para imprimir en el depósito.[34]

8.2. Gestión de Agenda (Servicios)

Para evitar la doble reserva.

- **Solución:** Base de datos propia con tabla bookings y validación estricta de solapamiento de fechas en el backend.
- **Alternativa:** Integración bidireccional con **Google Calendar API** o **Calendly API**. Cuando se reserva en el sitio, se bloquea en el calendario personal del terapeuta.

9. Conclusión y Hoja de Ruta

Construir una plataforma de terapia holística al nivel de los "grandes desarrolladores" es un ejercicio de arquitectura de software, no solo de diseño web. Requiere abandonar las soluciones prefabricadas limitadas y abrazar un stack tecnológico moderno (**Next.js, Supabase, Mercado Pago**) que permita la flexibilidad necesaria para manejar modelos de negocio híbridos.

El uso de **Ingeniería Agéntica (Google Antigravity)** democratiza este nivel de desarrollo. Al actuar como arquitecto y proveer los *prompts* detallados y estructurales definidos en este reporte, es posible generar un código base robusto, seguro y escalable que, tradicionalmente, requeriría un equipo completo de ingenieros. El resultado final no será solo una tienda, sino un ecosistema digital inmersivo que fusiona contenido, comunidad y comercio, estableciendo una marca de autoridad en el mercado del bienestar.

Referencias Integradas

Este reporte sintetiza análisis de mercado de sitios como *The Hoodwitch* y *Labyrinthos* [3], documentación técnica de *Next.js* y *Mercado Pago* [13], y metodologías de desarrollo agéntico con *Google Antigravity*.[37] Las estrategias de diseño y base de datos se basan en patrones de arquitectura de software empresarial y diseño UX específico para e-commerce.[1]

Obras citadas

1. What Is SOA (Service-Oriented Architecture)? - Oracle, fecha de acceso: enero 26, 2026, https://www.oracle.com/service-oriented-architecture-soa/
2. What is Headless eCommerce? Guide to Decoupled Architecture - KIBO, fecha de acceso: enero 26, 2026, https://kibocommerce.com/resource-center/headless-ecommerce/
3. The Hoodwitch, fecha de acceso: enero 26, 2026, https://www.thehoodwitch.com/
4. interviews — Blog — The Hoodwitch, fecha de acceso: enero 26, 2026, https://www.thehoodwitch.com/blog/tag/interviews
5. The Hoodwitch Is Changing the Face of Modern Witchcraft - VICE, fecha de acceso: enero 26, 2026, https://www.vice.com/en/article/the-hoodwitch-is-changing-the-face-of-modern-witchcraft/
6. Featured — The Hoodwitch, fecha de acceso: enero 26, 2026, https://www.thehoodwitch.com/featured
7. Labyrinthos Academy: Learning Tarot with Monsters of Lore, fecha de acceso: enero 26, 2026, https://labyrinthos.co/pages/frontpage
8. Labyrinthos Academy - Online Tarot School for Witches & Wizards, fecha de acceso: enero 26, 2026, https://labyrinthos.co/

9. What is Smudging and How do I Smudge? - Sage Goddess, fecha de acceso: enero 26, 2026, https://www.sagegoddess.com/how-do-i-smudge/
10. Sage Theme Usage Statistics - BuiltWith Trends, fecha de acceso: enero 26, 2026, https://trends.builtwith.com/framework/Sage-Theme
11. 10 Software Architecture Patterns You Must Know About - Simform, fecha de acceso: enero 26, 2026, https://www.simform.com/blog/software-architecture-patterns/
12. How to Create a Database Schema for an E-Commerce Application? - Tutorials - Back4app, fecha de acceso: enero 26, 2026, https://www.back4app.com/tutorials/how-to-create-a-database-schema-for-an-e-commerce-application
13. Building Ecommerce Sites with Next.js and Shopify | Vercel Knowledge Base, fecha de acceso: enero 26, 2026, https://vercel.com/kb/guide/building-ecommerce-sites-with-next-js-and-shopify
14. Building a Modern E-commerce System: Next.js 15+ Shopify + Go High Level Architecture 2/2 - Medium, fecha de acceso: enero 26, 2026, https://medium.com/@d_pt_m/building-a-modern-e-commerce-system-next-js-15-shopify-go-high-level-architecture-55b19ca23465
15. Building a Modern E-commerce App with Next.js 15, Zustand, and TypeScript, fecha de acceso: enero 26, 2026, https://dev.to/blamsa0mine/building-a-modern-e-commerce-app-with-nextjs-15-zustand-and-typescript-18i5
16. Building a Production-Ready E-Commerce Platform with Next.js 16: A Complete Guide, fecha de acceso: enero 26, 2026, https://medium.com/@reactjsbd/building-a-production-ready-e-commerce-platform-with-next-js-16-a-complete-guide-48b9dedbb8fb
17. How to Build a Digital Products Store with Medusa and Next.js - freeCodeCamp, fecha de acceso: enero 26, 2026, https://www.freecodecamp.org/news/build-a-digital-products-store-with-medusa-and-next-js/
18. Building a store for digital products with Next.js and Medusa - DEV Community, fecha de acceso: enero 26, 2026, https://dev.to/medusajs/building-a-store-for-digital-products-with-nextjs-and-medusa-1o6m
19. Understanding state management in Next.js - LogRocket Blog, fecha de acceso: enero 26, 2026, https://blog.logrocket.com/guide-state-management-next-js/
20. WooCommerce vs Shopify, fecha de acceso: enero 26, 2026, https://woocommerce.com/woocommerce-vs-shopify/
21. WooCommerce vs Shopify: A Closer Look at the Showdown (2023) - Saucal, fecha de acceso: enero 26, 2026, https://saucal.com/blog/woocommerce-vs-shopify/
22. Shopify vs. WooCommerce, fecha de acceso: enero 26, 2026, https://www.shopify.com/compare/shopify-vs-woocommerce
23. Ecommerce - tiendanube vs shopify vs woocommerce : r/NegociosArgentina - Reddit, fecha de acceso: enero 26, 2026,

https://www.reddit.com/r/NegociosArgentina/comments/1mou6lb/ecommerce_tiendanube_vs_shopify_vs_woocommerce/?tl=en
24. How to Design a Relational Database for E-commerce Website - GeeksforGeeks, fecha de acceso: enero 26, 2026, https://www.geeksforgeeks.org/dbms/how-to-design-a-relational-database-for-e-commerce-website/
25. Designing the database schema for a new e-commerce platform and considering factors like normalization, denormalization, indexing, and performance optimization to ensure efficient storage and retrieval of data. - Biswanath Giri, fecha de acceso: enero 26, 2026, https://bgiri-gcloud.medium.com/designing-the-database-schema-for-a-new-e-commerce-platform-and-considering-factors-like-ec28d4fb81db
26. Cart organization - should I combine delivery and collection together or keep them separate?, fecha de acceso: enero 26, 2026, https://ux.stackexchange.com/questions/57034/cart-organization-should-i-combine-delivery-and-collection-together-or-keep-th
27. Add a Shopping Cart & Checkout to a Next.js App with Snipcart - YouTube, fecha de acceso: enero 26, 2026, https://www.youtube.com/watch?v=o5wT5XoPUSo
28. Preferences - Advanced features - Mercado Pago Developers, fecha de acceso: enero 26, 2026, https://www.mercadopago.com.co/developers/en/docs/checkout-bricks/wallet-brick/advanced-features/preferences
29. How to integrate checkout in marketplace - How-tos - Mercado Pago Developers, fecha de acceso: enero 26, 2026, https://www.mercadopago.com.ar/developers/en/docs/checkout-pro/how-tos/integrate-marketplace
30. Create and configure a payment preference - Stages of integration - Mercado Pago, fecha de acceso: enero 26, 2026, https://www.mercadopago.com.ar/developers/en/docs/checkout-pro/create-payment-preference
31. Mercado Pago Wallet - Integration configuration, fecha de acceso: enero 26, 2026, https://www.mercadopago.com.ar/developers/en/docs/checkout-api-payments/integration-configuration/integrate-mp-wallet
32. Delivering digital goods using Next.js and Stripe Checkout? - Reddit, fecha de acceso: enero 26, 2026, https://www.reddit.com/r/stripe/comments/1c1xljr/delivering_digital_goods_using_nextjs_and_stripe/
33. Correo Argentino-Plataforma de Integración- API 2.0, fecha de acceso: enero 26, 2026, https://www.correoargentino.com.ar/MiCorreo/public/img/pag/apiPaqAr-v2.pdf
34. API docs - Andreani Developers, fecha de acceso: enero 26, 2026, https://developers.andreani.com/document
35. Web API Docs | API Reference | Epackenvios, fecha de acceso: enero 26, 2026, https://api.epackenvios.com/

36. Autenticación - Envíopack, fecha de acceso: enero 26, 2026, https://developers.enviopack.com.ar/autenticacion
37. Getting Started with Google Antigravity, fecha de acceso: enero 26, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
38. Build with Google Antigravity, our new agentic development platform, fecha de acceso: enero 26, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
39. Google Antigravity Tutorial: Build a Finance Risk Dashboard - DataCamp, fecha de acceso: enero 26, 2026, https://www.datacamp.com/tutorial/google-antigravity-tutorial
40. 24 Aesthetic Colour Palettes for Soulful, Boho & Spiritual Brands (with HEX Codes), fecha de acceso: enero 26, 2026, https://thebrandalchemists.com/blogs/news/holistic-brand-color-palettes-2025
41. 9 Luxury Color Palettes That Define High-End Design in 2025 - Brandlic, fecha de acceso: enero 26, 2026, https://brandlic.studio/9-luxury-color-palettes-that-define-high-end-design-in-2025/
42. Create preference for multiple items - Additional settings - Mercado Pago Developers, fecha de acceso: enero 26, 2026, https://www.mercadopago.cl/developers/en/docs/checkout-pro/additional-settings/preference-multiple-items
43. Best Prompts for Coding Inside Antigravity (Top 50) - Skywork.ai, fecha de acceso: enero 26, 2026, https://skywork.ai/blog/agent/best-prompts-antigravity/
44. Best Free Canva Fonts for Holistic & Soul-Led Brands (2025), fecha de acceso: enero 26, 2026, https://thebrandalchemists.com/blogs/news/best-free-canva-fonts-holistic-brands
45. 8 Free Fonts for Wellness Brands - The Denizen Co., fecha de acceso: enero 26, 2026, https://www.thedenizenco.com/journal/8-free-fonts-for-the-wellness-brand
46. Tarot Card API - CodeSandbox, fecha de acceso: enero 26, 2026, http://codesandbox.io/p/github/DudeDrew/tarot-api
47. krates98/tarotcardapi: Free Tarot Card API: A versatile and user-friendly API offering random tarot card selection and a complete list of cards with high-quality images. Ideal for integration in tarot reading apps, spiritual websites, and educational projects. Open-source and ready for collaboration and use. - GitHub, fecha de acceso: enero 26, 2026, https://github.com/krates98/tarotcardapi
48. Moon API Parameters - Meteomatics, fecha de acceso: enero 26, 2026, https://www.meteomatics.com/en/api/available-parameters/moon/
49. Moon Phase | AstronomyAPI, fecha de acceso: enero 26, 2026, https://docs.astronomyapi.com/endpoints/studio/moon-phase
50. Best Cart customization Apps For 2026 - Shopify App Store, fecha de acceso: enero 26, 2026,

https://apps.shopify.com/categories/marketing-and-conversion-checkout-cart-customization/all
51. Integration model - Checkout API (via Orders) - Mercado Pago Developers, fecha de acceso: enero 26, 2026, https://www.mercadopago.com.ar/developers/en/docs/checkout-api-orders/integration-model